

此页在源站不可用

第 1 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 2 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 3 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 4 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 5 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 6 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 7 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 8 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 9 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 10 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 11 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 12 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 13 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 14 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 15 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 16 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 17 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 18 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 19 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 20 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 21 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 22 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 23 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 24 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 25 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 26 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 27 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 28 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 29 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 30 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 31 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 32 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 33 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 34 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 35 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 36 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 37 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 38 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 39 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 40 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 41 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 42 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 43 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 44 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 45 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 46 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 47 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 48 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 49 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

此页在源站不可用

第 50 页 / 共 127 页

雨课堂服务端该页图片签名已过期，网页端同样无法显示
页序已保留，不影响其余页面的对应关系

思考：

- ☁ 1. 下面的代码运行后，输出结果是什么？在Jupyter中运行验证。

```
list1=["revenue","income","cost"]  
list2=[10,20]  
list3=[list2,list1]  
print(list3[1][1])
```



参考答案：

☁ 1. 输出结果：“income”



列表的比较:

- 在调用比较方法时, 需要使用operator模块, 通过import方法导入, 调用operator.eq方法来进行运算
- 右图示例中, operator.eq(list1,list2)就是将列表list1与list2比较, 如果两者相同, 就返回True, 如果不同, 就返回False。

```
# 导入 operator 模块
import operator

list1 = [10, "income"]
list2 = [20, "revenue"]
list3 = [20, "revenue"]
print("list1与list2比较: ", operator.eq(list1, list2))
print("list2与list3比较: ", operator.eq(list2, list3))
```

```
list1与list2比较: False
list2与list3比较: True
```

编码练习：

- 1. 参照本节的语句，在Jupyter中将示例代码输入，运行后查看运行结果，熟悉列表比较的相关知识点。



思考：

- ☁ 1. 下面的代码运行后，输出结果是什么？在Jupyter中运行验证。

```
# 导入 operator 模块
import operator

list1 = ["book", 20]

list1[0]=20

list2 = [20]*2

print(operator.eq(list1,list2))
```

参考答案:

☁ 1. 输出结果: True



元组与列表类似，但元组中的元素不可修改

元组的位置索引：

- 如果从左往右计数，元组元素的索引值为从0开始。
- 如果从右往左计数，元组元素的索引值为从-1开始。
- 针对Unicode（如中文），一个元素也按一个索引计数，参见右图的中文元组。

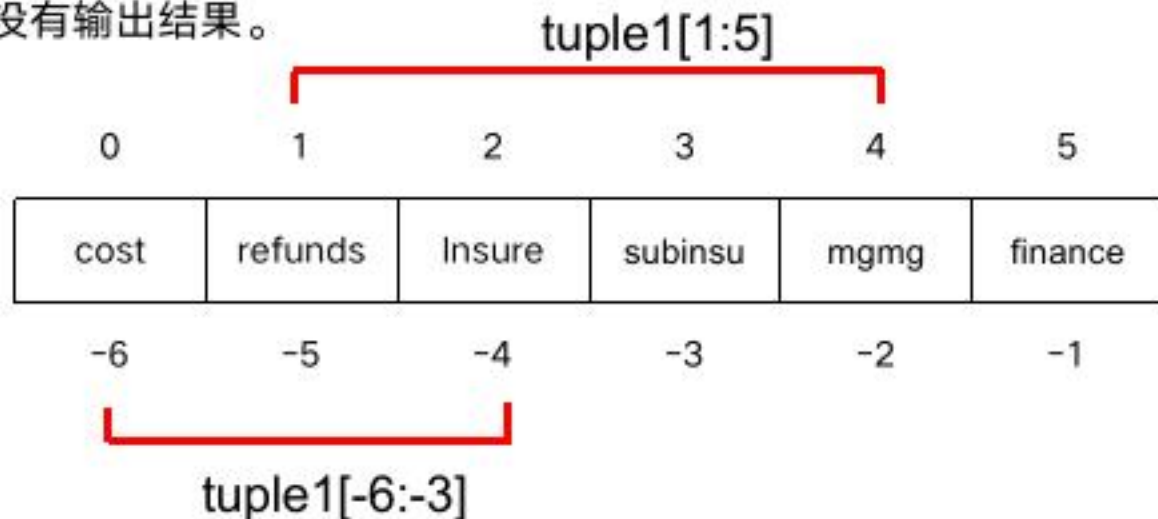
0	1	2	3	4	5
cost	refunds	Insure	subinsu	mgmg	finance
-6	-5	-4	-3	-2	-1
0	1	2	3	4	5
营业收入	利息收入	已赚保费	营业成本	退保金	研发费用
-6	-5	-4	-3	-2	-1

2.3元组-访问元组中的元素



元组的元素访问:

- 要访问元组中某个位置的元素, 用`tuple[i]`。用法示例: 针对上图第一个元组
`tuple1=("cost","refunds","Insure","subinsu","mgmg","finance")`, 要访问第2个元素, 用`tuple1[1]`; 要访问倒数第3个元素, 用`tuple1[-3]`
- 要访问元组中某一段位置的元素, 用`tuple[i:j]`。用法示例: 对元组
`tuple1=("cost","refunds","Insure","subinsu","mgmg","finance")`, 要访问第2-5个元素, 用`tuple1[1:5]`, 结果为
`("refunds","Insure","subinsu","mgmg")`。注意此处是算头不算尾, 1对应的"refunds"在输出结果中, 但5对应的"finance"没有输出。
- 只能从左往右访问元组中某一段位置的字符, 不能从右往左访问。示例说明: `tuple1[-6:-3]`输出结果为
`("cost","refunds","Insure")`, 但`tuple1[-3:-6]`就没有输出结果。



元组的元素访问的示例代码：

- ☞ Tuple1[-3:-6]：是从右向左访问，因此没有输出结果。
- ☞ Tuple1[:2]：省略了开始位置的值，访问的是元组tuple1第一个元素到第2个元素，因此为('cost', 'refunds')。
- ☞ tuple1[-2:]：省略了结束位置的值，访问的是元组tuple1倒数第2个元素到最后一个元素，因此为('mgmg', 'finance')。
- ☞ tuple1[:]: 省略了开始位置的值和结束位置的值，访问的是元组中所有元素，因此就输出所有元素。

#tuple位置索引示例

```
tuple1=("cost","refunds","Insure","subinsu","mgmg","finance")
print(tuple1[1],tuple1[-3])
print(tuple1[1:5])
print(tuple1[-6:-3])
print(tuple1[-3:-6])
print(tuple1[:2])
print(tuple1[-2:])
print(tuple1[:])
```

```
refunds subinsu
('refunds', 'Insure', 'subinsu', 'mgmg')
('cost', 'refunds', 'Insure')
()
('cost', 'refunds')
('mgmg', 'finance')
('cost', 'refunds', 'Insure', 'subinsu', 'mgmg', 'finance')
```

编码练习：

- 1. 参照本节的语句，在Jupyter中将示例代码输入，运行并查看运行结果，熟悉元组访问的相关知识点。



思考：

- ☁ 1. 下面的代码运行后，输出结果是什么？在Jupyter中运行验证。

```
tuple2=("营业成本","退保金","", "分保费用","销售费用","管理费用","财务费用")  
print(tuple2[-4:4])
```



参考答案：

- 1. 结果：('分保费用',)
- 当元组中只有一个元素时，需要在末尾加逗号，否则系统会认为是字符串类型，而不是元组类型。



- 元组中的元素是不能修改的，但元组作为整个对象是可以修改的
- `tuple1=tuple1+tuple2`是对`tuple1`进行修改，将`tuple1`和`tuple2`进行组合，组合后再赋值给`tuple1`，因此最后输出的`tuple1`的值发生了改变。
- `id(tuple1)`可以查到`tuple1`在内存中的地址。可以看到，`tuple1`修改前的`id`与修改后的`id`是不一样的，说明这已经是两个`tuple1`对象，虽然两者的名字完全相同。

```
#修改tuple1整个对象
tuple1=("cost","refunds","Insure")
print("修改前id:",id(tuple1))
tuple2=("营业成本","退保金")
tuple1=tuple1+tuple2
print(tuple1)
print("修改后id:",id(tuple1))
```

```
修改前id: 2138700004608
('cost', 'refunds', 'Insure', '营业成本', '退保金')
修改后id: 2138706271264
```

- 元组中的元素是不能删除的，但元组作为整个对象是可以删除的。
- del tuple1是将tuple1对象整个删除，因此内存中不再存在该对象。后续用print函数调用tuple1时就报错

```
#1-3 2.3元组的删改 示例2  
#删除tuple1整个对象  
tuple1=("cost","refunds","Insure")  
del tuple1  
print(tuple1)
```

```
Traceback (most recent call last)  
~\AppData\Local\Temp\ipykernel_20704\114355516.py in <module>  
      3 tuple1=("cost","refunds","Insure")  
      4 del tuple1  
----> 5 print(tuple1)
```

NameError: name 'tuple1' is not defined

编码练习：

- 1. 参照本节的语句，在Jupyter中将示例代码输入，运行并查看运行结果，熟悉元组删改的相关知识点。



思考:

- ☁ 1. 下面的代码运行后，输出结果是什么？在Jupyter中运行验证。

```
tuple1=("cost","refunds","Insure")  
tuple1[0]="asset"  
print(tuple1)
```

- ☁ 2. 下面的代码运行后，输出结果是什么？在Jupyter中运行验证。

```
tuple1=("cost","refunds","Insure")  
tuple2=("营业成本","退保金")  
del tuple2  
tuple1=tuple1+tuple2  
print(tuple1)
```

参考答案:

- 1. 结果: 系统会报错, 因为元组中的元素值不允许修改, 但`tuple1[0]`却想修改第1个元素的值。
- 2. 系统报错, 因为`tuple2`对象已经删除, `tuple1=tuple1+tuple2`就无法运算。



- 元组常用的运算有：+、*、in、len()
- 乘法运算式('Inc',) * 4中，单个元素的元组写为('Inc',)，而不是('Inc')。如果写成('Inc')，系统会识别为str字符串类型，而不是元组类型

说明	运算式	结果
+	(x, y, z) + (0, 1, 2)	(x, y, z, 0, 1, 2)
*	('Inc',) * 4	('Inc','Inc','Inc','Inc')
in	3 in (1, 2, 3)	True
len()	len(1,2,3)	3



元组的运算：

- ⌞ +号是将两个元组的元素进行组合
- ⌞ *号是将元组的元素重复多次，因此最后输出的元组中包含多个相同的元素

#元组的+运算示例

```
tuple1=("cost","refunds","Insure")  
tuple2=("营业成本","退保金")  
tuple1=tuple1+tuple2  
print(tuple1)
```

('cost', 'refunds', 'Insure', '营业成本', '退保金')

*#元组的*运算示例*

```
tuple1=('营业收入','利息收入')  
tuple2=(200,)*3  
tuple3=tuple1+tuple2  
print(tuple3)
```

('营业收入', '利息收入', 200, 200, 200)

编码练习：

- 1. 参照本节的语句，在Jupyter中将示例代码输入，运行后查看运行结果，熟悉元组运算的相关知识点。



思考:

- ☁ 1. 下面的代码运行后, 输出结果是什么? 在Jupyter中运行验证。

```
tuple2=(200)*3  
print(tuple2)  
tuple2=(200,)*3  
print(tuple2)
```



参考答案:

- 1. 输出结果分别是: 600 和 (200, 200, 200)。分析如下:
- $\text{tuple2}=(200)*3$ 中, (200)被系统识别为整数, 因此结果为600。
- $\text{tuple2}=(200,)*3$ 中, (200,)被系统识别为元组, 因此结果为(200, 200, 200)。



创建字典有两种方法：

- ☞ 用{ }来创建，系统会默认为字典类型。
- ☞ 用dict()函数创建集合，在括号中可以是赋值运算式、映射函数方式、可迭代对象方式。
- ☞ 右图，dict1={}是用{}直接创建一个空字典，用type(dict1)函数可显示该变量的类型为dict。
- ☞ dict(Income=30, Asset=200, cost=20)是用dict()来创建一个字典，字典中的键值对用三个表达式来生成，转换为{'Income': 30, 'Asset': 200, 'cost': 20}。
- ☞ dict([('one', 1), ('two', 2), ('three', 3)])也是用dict()来创建一个字典，字典中的键值对用列表[('one', 1), ('two', 2), ('three', 3)]来生成，转换为{'one': 1, 'two': 2, 'three': 3}。

#创建字典的示例

```
dict1={}
print(dict1, type(dict1))
dict2=dict(Income=30, Asset=200, cost=20)
print(dict2)
dict3=dict([('one', 1), ('two', 2), ('three', 3)])
print(dict3)
```

```
{ } <class 'dict'>
{'Income': 30, 'Asset': 200, 'cost': 20}
{'one': 1, 'two': 2, 'three': 3}
```

字典中的元素特点:

- 字典内的每对元素由key-value组成，每对元素用逗号分隔，表示为：`dict{key:value, key:value, ... }`
- key必须用不可变元素，也就是字符串、数字、元组等，不能用列表、集合。
- key值不要重复，如果重复，系统只会记住最后一个。



字典的元素示例代码：

图1，在dict1中，有一个key值为34，其值为89.2，说明用数字作为key是允许的。在实际编程中，这种情况一般是将key值作为元素值的序号来使用。

图2，在dict1中，定义了一个key为列表[“营业收入”]，此时系统报错，因为列表为可变类型，不能作为key。

图3，在dict1中，定义了两个相同的key“营业收入”，值并不相同。但是，在输出时，可以看到，系统只记录了最后一个key“营业收入”及其值89.2，因此在构建字典时，key不要重复，不然会导致数据丢失或者错误。

```
#字典的key值示例
dict1={"营业收入":2000,"销售费用":300,34:89.2}
print(dict1)
```

```
{'营业收入': 2000, '销售费用': 300, 34: 89.2}
```

图1

```
#字典的key值示例
dict1=[["营业收入"]:2000,"销售费用":300,34:89.2}
print(dict1)
```

```
TypeError                                 Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_23820\1504803821.py in <module>
      1 #字典的key值示例
----> 2 dict1=[["营业收入"]:2000,"销售费用":300,34:89.2}
      3 print(dict1)
```

```
TypeError: unhashable type: 'list'
```

图2

```
#字典的key值示例
dict1={"营业收入":2000,"销售费用":300,"营业收入":89.2}
print(dict1)
```

```
{'营业收入': 89.2, '销售费用': 300}
```

图3

编码练习：

- 1. 参照本节的语句，在Jupyter中将示例代码输入，运行并查看运行结果，熟悉字典元素的相关知识点。



思考:

- ☁ 1. 下面的代码运行后，输出结果是什么？在Jupyter中运行验证。

```
dict1 = {('收入','成本'): 3230, "费用": 100}  
print(dict1)
```



参考答案:

- ☁ 1. 结果: `{('收入', '成本'): 3230, '费用': 100}`, 这说明字典中key可以用元组类型。



访问字典的值：

- 访问字典的值时，需要将相应的key放入到方括号中。
- 下图中，可以看到dict1["销售费用"]就是通过key“销售费用”来访问其对应的值

```
dict1={"营业收入":2000,"销售费用":300,"研发费用":89.2}  
print(dict1["销售费用"])
```

300

- 下图中，dict1["财务费用"]是访问key“财务费用”对应的值，但是“财务费用”在字典的key中并不存在，因此系统报错。

```
dict1={"营业收入":2000,"销售费用":300,"研发费用":89.2}  
print(dict1["财务费用"])
```

```
KeyError                                Traceback (most recent call last)  
~\AppData\Local\Temp\ipykernel_23820\501694928.py in <module>  
      1 dict1={"营业收入":2000,"销售费用":300,"研发费用":89.2}  
----> 2 print(dict1["财务费用"])
```

KeyError: '财务费用'

编码练习：

- 1. 参照本节的语句，在Jupyter中将示例代码输入，运行并查看运行结果，熟悉访问字典的值的相关知识点。



思考:

- ☁ 1. 下面的代码运行后，输出结果是什么？在Jupyter中运行验证。

```
dict1={1:"营业收入",2:"销售费用",3:"研发费用"}  
print(dict1[3])
```



参考答案：

☁ 1. 结果：研发费用。



字典元素的新增：

- 往字典中新增元素就是直接增加新的键/值对。
- 下图中，在执行dict1[“研发费用”]=89.2时，系统先会查找dict1中是否有key“研发费用”，如果没有，就将新的键/值对{‘研发费用’:89.2}添加到dict1中，因此最后输出的dict1中包含了三个键值对。

```
dict1={"营业收入":2000,"销售费用":300}  
dict1["研发费用"]=89.2  
print(dict1)
```

{'营业收入': 2000, '销售费用': 300, '研发费用': 89.2}

字典元素的修改：

- 修改字典中的值，只需要给key对应的值重新赋值即可。
- 下图中，在执行dict1["销售费用"]=500时，字典会查找现有内容中是否有key等于“销售费用”，一旦查到，就会更新其值为500，因此最后输出的dict1中“销售费用”的值为500

```
dict1={"营业收入":2000,"销售费用":300}  
dict1["销售费用"]=500  
print(dict1)
```

```
{'营业收入': 2000, '销售费用': 500}
```

字典元素的删除：

- 移除字典中的一个键/值对，可用pop方法。
- 清空字典中的所有键/值对，可用clear方法。

删除字典对象：

- 删除整个字典对象，可直接用del命令来对象。删除后，系统将不能再访问该字典对象。



字典元素的删除示例-pop()方法:

- 在执行dict1.pop("销售费用")时,就是将键值对{"销售费用":300}从dict1中移除,因此最后输出的结果就只有一个键值对。

```
dict1={"营业收入":2000,"销售费用":300}  
dict1.pop("销售费用")  
print(dict1)
```

{'营业收入': 2000}

字典元素的删除示例-clear()方法:

- 在执行dict1.clear()时,将整个dict1的键值对全部清空,因此最后输出的字典结果就为空。

```
dict1={"营业收入":2000,"销售费用":300}  
dict1.clear()  
print(dict1)
```

{}

字典对象的删除-del命令:

- ☁ 在执行del dict1时, 将dict1对象删除, 该对象将不再存在, 因此在后面访问dict1时就会报错。

```
dict1={"营业收入":2000,"销售费用":300}  
del dict1  
print(dict1)
```

```
Traceback (most recent call last)  
~\AppData\Local\Temp\ipykernel_23820\1841519751.py in <module>  
      1 dict1={"营业收入":2000,"销售费用":300}  
      2 del dict1  
----> 3 print(dict1)  
  
NameError: name 'dict1' is not defined
```

编码练习：

- 1. 参照本节的语句，在Jupyter中将示例代码输入，运行并查看运行结果，熟悉字典增删改的相关知识点。



思考:

- ☁ 1. 已知dict1={"Income":100,"cost":30}, 现增加新键值对{ 'asset': 200}, 又更新键“cost”的值为50, 移除键“Income”及其值, 编写代码, 并回答最终输出结果是什么?



参考答案:

1. 输出结果: {'cost': 50, 'asset': 200}。

```
dict1={"Income":100,"cost":30}  
dict1["asset"]= 200  
dict1['cost']=50  
dict1.pop("Income")  
print(dict1)
```



集合用一对大括号{ }表示，用于包含一组无序的对象。

创建集合：

- ☞ 用set()函数创建集合，在括号中的数据类型可以是字符串、列表、元组、字典。
- ☞ 用{ }来创建，要求其中至少包含一个元素，不然系统会默认为字典类型，而不是集合类型。



创建集合示例：

- set("营业收入")是将字符串转化为集合，转化后每个字都作为一个元素
- set({"收入":20,"资产":300})是将字典转化为集合，转化后仅保留key部分作为元素
- set3={"Income"}是直接创建一个集合，要求其中至少有一个元素。如果不添加元素，系统会默认为字典类型，而不是set类型。
- type(set3)为type()函数，用于判断变量的类型。从输出结果可以看出，set3的类型为set。

```
set1=set("营业收入")
print(set1)
set2=set({"收入":20,"资产":300})
print(set2)
set3={"Income"}
print(set3,type(set3))
```

```
{'营', '收', '入', '业'}
{'收入', '资产'}
{'Income'} <class 'set'>
```

编码练习：

- 1. 参照上面的语句，在Jupyter中输入代码并运行，查看运行结果，并熟悉创建集合的相关知识点。



思考:

- ☞ 1. 下面代码的输出结果是什么 () ?

```
set2=set(("收入","资产"))  
print(set2)
```

- ☞ 2. 下面代码中的set2和set3是什么数据类型 () ?

```
set2={};set3={"revenue"}  
print(type(set2),type(set3))
```



参考答案:

- 1. 输出结果: {'收入', '资产'}
- 2. set2为dict类型; set3为set类型



创建集合示例：

- ☞ 集合中的元素是无序排列的，因此无法通过位置索引进行访问。
- ☞ 集合中的元素不能重复。如果重复，系统默认只保存一个。



集合元素特点示例：

- ☁ 集合set1在设置时元素的位置与输出时元素的位置是不一样的，元素的位置在集合内部是无序存放的。

```
set1={'Income', 'Asset', 'Cost', 'Revenue'}  
print(set1)  
  
{'Income', 'Revenue', 'Cost', 'Asset'}
```

- ☁ 有两对元素是重复的。在输出时，系统自动将重复的元素删除，只保留了四个元素。

```
set1={'Income', 'Asset', 'Income', 'Cost', 'Revenue', 'Cost'}  
print(set1)  
  
{'Income', 'Revenue', 'Cost', 'Asset'}
```

编码练习：

- 1. 参照上面的语句，在Jupyter中输入代码并运行，查看运行结果，并熟悉集合元素特点的内容。



思考:

- ☁ 1. 下面代码的输出结果是什么 () ?

```
set1={'Income', 'Asset', 'Income','Cost', 'Revenue','Cost'}  
print(set1[2])
```



参考答案:

1. 系统会报错，因为集合元素不能按索引位置访问。



添加元素:

- ☞ 往集合中新增元素可以用`add()`、`update()`两种方法。
- ☞ 如果`x`为元素，使用`add(x)`方法。如果该元素已经存在于集合中，则不做任何操作。
- ☞ 如果想`x`为集合、列表、元组、字典，可以直接使用`update(x)`更新。



添加集合元素-add()方法:

- ☞ `set(("营业收入", "利息收入", "已赚保费"))`是用`set()`函数将元组转换为集合类型。
- ☞ `set1.add("利息收入")`是将“利息收入”添加到集合中。但因为原集合中已有该元素，故可以看到集合元素没有改变。
- ☞ `set1.add("其他业务收入")`是将“其他业务收入”添加到集合中。原集合中没有该元素，故从输出结果看出，该元素在集合中已新增。

```
#集合新增元素示例-add
set1 = set(("营业收入", "利息收入", "已赚保费"))
set1.add("利息收入")
print(set1)
set1.add("其他业务收入")
print(set1)
```

```
{'营业收入', '利息收入', '已赚保费'}
{'营业收入', '利息收入', '已赚保费', '其他业务收入'}
```

添加集合元素-update()方法:

- set1.update({1,2})是将集合{1,2}中的两个元素添加到set1中, 因此在输出结果中可以看到已新增这两个元素。
- set2.update([3,4],(5,6))是将列表[3,4]和元组(5,6)添加到se2中, 因此在输出结果中可以看到这四个元素是打散后新增到集合中的, 原有的列表、元组已不存在了。

```
#新增集合元素示例-update
set1 = set(("营业收入", "利息收入", "已赚保费"))
set1.update({1, 2})
print(set1)
set2 = {"Income", "Interest", "Revenue"}
set2.update([3, 4], (5, 6))
print(set2)
```

{'营业收入', '利息收入', 1, 2, '已赚保费'}
{3, 'Interest', 4, 5, 6, 'Income', 'Revenue'}

移除集合元素方法：

- ⇒ 对集合的元素进行删除，有四种方法：`remove()`、`discard()`、`pop()`、`clear()`。
- ⇒ `remove()`方法是将元素从集合中移除，如果元素不存在，则会发生错误。
- ⇒ `discard()`方法也是将指定的元素移除。不过，如果元素不存在，系统不会报错。
- ⇒ `pop()`方法是在集合随机排序后，删除第一个元素。
- ⇒ `clear()`是将集合的元素都清空。



移除集合元素-remove()方法:

- set2.remove("Income")查找到元素“Income”后, 将其删除, 因此最终的集合中没有“Income”。

```
#集合remove()方法-移除存在的元素
set2 = {"Income", "Interest", "Revenue"}
set2.remove("Income")
print(set2)

{'Interest', 'Revenue'}
```

- set2.remove("cost")将元素“cost”删除。

- 执行set2.remove("cost")时, 查找不到元素“cost”, 因此系统报错。

```
#集合remove()方法-移除不存在的元素
set2 = {"Income", "Interest", "Revenue"}
set2.remove("cost")
print(set2)
```

```
KeyError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_23820\3069382982.py in <module>
      1 set2 = {"Income", "Interest", "Revenue"}
--> 2 set2.remove("cost")
      3 print(set2)

KeyError: 'cost'
```

移除集合元素-discard()方法:

- set2.discard("Income")查找到元素“Income”后, 将其删除, 因此最终的集合中没有“Income”。set2.discard("cost")查找不到元素“cost”, 但不会报错。

```
#集合discard()方法-移除存在的元素
set2 = {"Income", "Interest", "Revenue"}
set2.discard("Income")
print(set2)
#集合discard()方法-移除不存在的元素
set2.discard("cost")
print(set2)

{'Interest', 'Revenue'}
{'Interest', 'Revenue'}
```

移除集合元素-pop()方法:

- set2.pop()是删除了集合set2随机排序后的第一个元素。

```
#集合pop()方法
set2 = {"Income", "Interest", "Revenue"}
print(set2)
set2.pop()
print(set2)

{'Income', 'Interest', 'Revenue'}
{'Interest', 'Revenue'}
```

移除集合元素-clear()方法:

☞ set2.clear()清除了集合的元素。所以在最后一行的输出为空。

```
#集合clear()方法
set2 = {"Income", "Interest", "Revenue"}
print(set2)
set2.clear()
print(set2)

{'Income', 'Interest', 'Revenue'}
set()
```



编码练习：

- 1. 参照上面的语句，在Jupyter中输入代码并运行，查看运行结果，并熟悉集合的增删改等内容。



思考:

- ☁ 1. 下面代码的输出结果是什么 () ? 可在Jupyter中输入后, 验证输出结果。

```
set2 = {"Income", "Interest", "Revenue"}  
list1=[3,4]  
tuple1={2,3}  
set2.update(list1,tuple1)  
print(set2)
```

- ☁ 2. 下面代码的输出结果是什么 () ? 可在Jupyter中输入后, 验证输出结果。

```
set2 = {"Income", "Interest", "Revenue"}  
set2.pop("Revenue")
```

参考答案：

- 1. 输出结果为：{2, 3, 'Interest', 4, 'Income', 'Revenue'}
- 2. 系统会报错，因为pop()是随机移除元素，不能指定具体的元素。



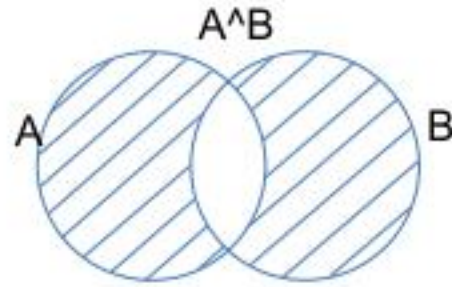
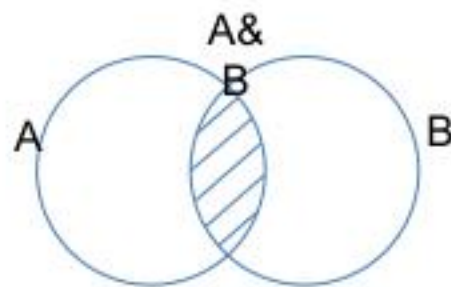
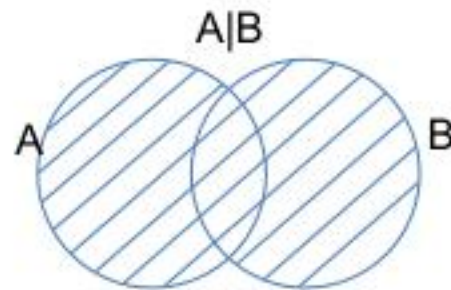
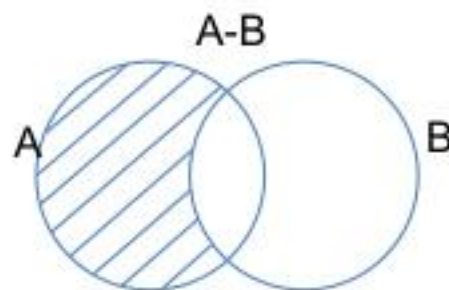
集合的主要运算包括相减 ($-$)、或 ($|$)、与 ($\&$)、异或 ($\wedge$) 等，右边为示意图

$A-B$: 是指集合A中包含，而集合B中不包含的元素

$A|B$: 是指包含于集合A或集合B中的元素

$A\&B$: 是指既包含与集合A又包含于集合B的元素

$A\wedge B$: 是指不同时包含于集合A和集合B的元素



集合运算示例:

- ⇒ $A=\text{set}(\text{"Income"})$ 中, $\text{set}()$ 是建立集合的函数, 将字符串“Income”拆散为单个字符, 去除重复的元素, 构成集合A。 $B=\text{set}(\text{"Revenue"})$ 功能相同, 不重复描述。
- ⇒ $A-B$: 以集合A为基准, 把在集合B中出现的元素剔除, 就得到相应结果。
- ⇒ $A|B$: 把集合A和B的元素合并, 并去除重复的元素, 得到相应的结果。
- ⇒ $A\&B$: 把集合A和B中都出现的元素选出来, 得到相应的结果。
- ⇒ $A^{\wedge}B$: 实际是 $A\&B$ 的补集, 把集合A和B中都出现的元素剔除, 得到相应的结果。

#集合运算示例

```
A=set("Income")
B=set("Revenue")
print("A=",A)
print("B=",B)
print("A-B为:",A-B)
print("A|B为:",A|B)
print("A&B为:",A&B)
print("A^B为:",A^B)
```

$A = \{'c', 'e', 'I', 'm', 'n', 'o'\}$

$B = \{'e', 'u', 'v', 'n', 'R'\}$

$A-B$ 为: $\{'c', 'o', 'm', 'I'\}$

$A|B$ 为: $\{'c', 'e', 'I', 'u', 'm', 'v', 'n', 'R', 'o'\}$

$A\&B$ 为: $\{'e', 'n'\}$

$A^{\wedge}B$ 为: $\{'c', 'I', 'u', 'm', 'v', 'R', 'o'\}$

集合运算示例：

- ☞ 判断集合的元素个数，用len()函数来判断
- ☞ 判断元素是否在集合中，用in运算符
- ☞ 下图为示例，len(set1)用于计算集合set1的元素个数
- ☞ str1 in set1用于判断字符串str1的值是否也存在于集合set1中

```
#1-3 2.5集合 示例2  
set1={"收入","资产","成本","负债"}  
print(len(set1))  
str1="成本"  
print(str1 in set1)
```

```
4  
True
```



编码练习：

- 1. 参照上面的语句，在Jupyter中输入代码并运行，查看运行结果，并熟悉集合的运算等内容。



思考:

☁ 1. 下面代码的输出结果是什么 () ? 可在 Jupyter 中输入后, 验证输出结果。

```
A=set("收入成长")
B=set("长期应收")
print("A=",A)
print("B=",B)
print("A-B为: ",A-B)
print("A|B为: ",A|B)
print("A&B为: ",A&B)
print("A^B为: ",A^B)
```

☁ 2. 下面代码的输出结果是什么 () ? 可在 Jupyter 中输入后, 验证输出结果。

```
A=set(("收入","资产","成本","负债"))
print("A=",A)
B=set("营业收入")
print("B=",B)
print("A-B为: ",A-B,"\n","A&B为: ",A&B)
```

参考答案:

1.输出结果为:

- ⊙ $A = \{\text{'入'}, \text{'长'}, \text{'收'}, \text{'成'}\}$
- ⊙ $B = \{\text{'收'}, \text{'期'}, \text{'应'}, \text{'长'}\}$
- ⊙ $A - B$ 为: $\{\text{'入'}, \text{'成'}\}$
- ⊙ $A|B$ 为: $\{\text{'收'}, \text{'应'}, \text{'成'}, \text{'期'}, \text{'长'}, \text{'入'}\}$
- ⊙ $A \& B$ 为: $\{\text{'收'}, \text{'长'}\}$
- ⊙ $A \wedge B$ 为: $\{\text{'应'}, \text{'成'}, \text{'期'}, \text{'入'}\}$

2.输出结果为:

- ⊙ $A = \{\text{'收入'}, \text{'资产'}, \text{'负债'}, \text{'成本'}\}$
- ⊙ $B = \{\text{'营'}, \text{'收'}, \text{'入'}, \text{'业'}\}$
- ⊙ $A - B$ 为: $\{\text{'收入'}, \text{'资产'}, \text{'负债'}, \text{'成本'}\}$
- ⊙ $A \& B$ 为: $\text{set}()$ (注: 此处为空集合)



1 案例背景

2 知识准备

3 任务要求

4 任务解析

查阅华盛科技的资产负债表和利润表，可以列出该公司连续4年的营业成本、年初存货、年末存货等数据，参见下表所示：

单位（亿元）

序号	第1年	第2年	第3年	第4年
营业成本	68.96	63.10	71.53	102
年初存货	22.90	29.03	25.32	36.82
年末存货	29.03	25.32	36.82	50.15

根据上面的数据，要计算华盛科技连续4年的存货周转率，并根据计算结果，分析华盛科技的库存周转率的变化情况。

具体要求如下：

- 要求利用元组类型来定义计算项目，利用列表类型来定义项目对应的数值
- 要求采用对列表元素的访问方法来构建计算公式
- 要求采用对元组元素的访问方法来输出计算公式的文字说明
- 要求对数据结果进行对比，自动输出简要的指标分析说明
- 要求输出的结果格式化，可自行定义格式



1 案例背景

2 知识准备

3 任务要求

4 任务解析

思考：

- 1. 根据上面的任务要求，查阅存货周转率的计算公式，理解存货周转率的含义。
- 2. 思考如何利用元组、列表来构建数据结构，并通过元组、列表来访问数据，同时构建计算公式？



参考答案：

- ☁ 1.存货周转率的计算公式：
- ☁ $\text{存货周转率} = \text{营业成本} / \text{平均库存} = \text{营业成本} / ((\text{期初库存} + \text{期末库存}) / 2)$
- ☁ 存货周转率用于反映存货的周转速度，即存货的流动性及存货资金占用量是否合理。

- ☁ 2.元组用于定义“营业成本”、“年初存货”、“年末存货”等计算项目；
- ☁ 列表可用于定义计算项目每个年度的值；
- ☁ 可通过元组的位置索引、列表的位置索引来访问，从而构建计算公式、公式文字说明。



参考代码：

```
#1-3 5任务实施 示例
#用元组构建计算项目
fItem=("营业成本","年初库存","年末库存")
#用列表构建项目的值
fValue1=[68.96, 22.9, 29.03]
fValue2=[63.10, 29.03, 25.32]
fValue3=[71.53, 25.32, 36.82]
fValue4=[102, 36.82, 50.15]

#通过列表索引的值构建计算公式
fIvt_trn1=fValue1[0]/((fValue1[1]+fValue1[2])/2)
fIvt_trn2=fValue2[0]/((fValue2[1]+fValue2[2])/2)
fIvt_trn3=fValue3[0]/((fValue3[1]+fValue3[2])/2)
fIvt_trn4=fValue4[0]/((fValue4[1]+fValue4[2])/2)

#通过元组索引的值显示公式说明
print("计算公式:")
print("    库存周转率={Item0}/(({Item1}+{Item2})/2)".\
      format(Item0=fItem[0], Item1=fItem[1], Item2=fItem[2]))
print("_____")
```



参考代码:

```
#格式化输出计算结果
print("计算结果:")
print("_____")
print("  第1年 | 第2年 | 第3年 | 第4年  ")
print("_____")
print("    ", "{:.2f}".format(fIvt_trn1), "| {:.2f}".format(fIvt_trn2), \
      " | {:.2f}".format(fIvt_trn3), \
      " | {:.2f}".format(fIvt_trn4))
print("_____")

#简单分析库存周转率的变化趋势
print("数据分析:")
if fIvt_trn2>fIvt_trn1:
    print("  第2年存货周转率为上升趋势")
else:
    print("  第2年存货周转率为下降趋势")
if fIvt_trn3>fIvt_trn2:
    print("  第3年存货周转率为上升趋势")
else:
    print("  第3年存货周转率为下降趋势")
if fIvt_trn4>fIvt_trn3:
    print("  第4年存货周转率为上升趋势")
else:
    print("  第4年存货周转率为下降趋势")
```



参考代码-输出结果:

计算公式:

库存周转率=营业成本/((年初库存+年末库存)/2)

计算结果:

第1年	第2年	第3年	第4年
2.66	2.32	2.30	2.35

数据分析:

第2年存货周转率为下降趋势

第3年存货周转率为下降趋势

第4年存货周转率为上升趋势



作业1：查阅华盛科技的资产负债表和利润表，可以列出该公司连续4年的销售收入、期初应收款、期末应收款等数据，参见下表所示：

单位（亿元）

序号	第1年	第2年	第3年	第4年
销售收入	110	95.63	119	163
期初应收款	36.66	45.02	39.31	41.52
期末应收款	45.02	39.31	41.52	58.86

根据上面的数据，要计算华盛科技连续4年的应收账款周转率，并根据计算结果，分析华盛科技的应收账款周转率的变化情况，具体要求如下：

- 要求利用字典类型来定义计算项目和对应的值
- 要求采用对字典键值对的访问方法来构建计算公式
- 要求采用对字典键的访问来输出计算公式的文字说明
- 要求对数据结果进行对比，自动输出简要的指标分析说明
- 要求输出的结果格式化，可自行定义格式



作业2：查阅华盛科技的资产负债表和利润表，可以列出该公司连续4年的销售收入、期初流动资产、期末流动资产等数据，参见下表所示：

单位（亿元）

序号	第1年	第2年	第3年	第4年
销售收入	110	95.63	119	163
期初流动资产	88.94	127	111	148
期末流动资产	127	111	148	189

根据上面的数据，要计算华盛科技连续4年的流动资产周转率，并根据计算结果，分析华盛科技的流动资产周转率的变化情况，具体要求如下：

- 要求利用集合类型来定义计算项目和对应的值
- 要求将集合转列表，对列表位置访问来构建计算公式
- 要求将集合转元组，对元组位置的访问来输出计算公式的文字说明
- 要求对数据结果进行对比，自动输出简要的指标分析说明
- 要求输出的结果格式化，可自行定义格式



感謝
ขอบคุณ
terima kasih
ありがとう
Thanks
谢谢